

RouteHub: A Low-Overhead, SDK-Native LLM Gateway for Agentic Workloads

Kye Gomez
The Swarm Corporation
kye@swarms.world

October 2026

Abstract

LLM gateways give applications one call signature for many model providers, and they sit on the critical path of every request. Agent workloads multiply the gateway’s own costs: dozens of sequential calls per task, short-lived worker processes, and wide fan-out. We present RouteHub, a gateway that keeps LiteLLM’s API but delegates transport, retries and response types to the official provider SDKs, loads nothing until a request needs it, pools clients per configuration and event loop, keeps connections alive across agent steps, and calls Anthropic’s native Messages API through a single-pass translator. We benchmark RouteHub against LiteLLM, any-llm, aisuite and the bare provider SDKs, with each library in its own environment against an instant mock server. RouteHub imports in 3.2 ms (LiteLLM: 1,235 ms) and reaches its first response in 267 ms, within 9 ms of the bare OpenAI SDK and 5.5× sooner than LiteLLM, with 3.9× less peak memory. Warm calls cost +9 μs more than the bare OpenAI SDK, where LiteLLM adds +647 μs. On the Anthropic route RouteHub is faster than the official SDK in sequential calls, streaming and async throughput. Over a real network, its 60 s keep-alive saves 67 ms on each call that follows a 10 s tool pause. RouteHub is open source at <https://github.com/The-Swarm-Corporation/RouteHub>.

1 Introduction

Applications that call large language models (LLMs) rarely talk to a single provider. Agent frameworks route cheap steps to small models, escalate hard ones to frontier models, and fail over between vendors when one is rate limited. Gateway libraries make this possible by exposing one call signature, usually the OpenAI Chat Completions format, and translating it for each provider. LiteLLM [4] popularised the approach; any-llm [16] and aisuite [17] followed with thinner designs.

A gateway sits on every request’s critical path, so its own costs are paid on every request. Those costs are easy to dismiss next to model latency, which is measured in seconds, but they compound in exactly the workloads where gateways are most used:

- **Agent loops.** A ReAct-style agent [24] alternates model calls with tool calls for tens to hundreds of steps. Per-call gateway work and any reconnection after a long tool call are paid at every step.
- **Short-lived processes.** Serverless functions, CLI tools, CI test suites and per-task worker processes pay the library’s import time on every start, the same problem SOCK [18] identified for Python packages in serverless platforms.
- **Fan-out.** Multi-agent systems issue many concurrent requests, so gateway CPU per request bounds throughput and inflates tail latency [6].
- **Supply chain.** Every transitive dependency is code that runs with the application’s credentials. In March 2026 two LiteLLM releases on PyPI carried a credential stealer, one of them via a .pth file that runs at interpreter start [15].

This paper presents RouteHub [9], the LLM gateway behind the Swarms agent framework. RouteHub keeps LiteLLM’s function names and module layout so migration is mostly an import change, but it is built around one principle: *do as little as possible between the caller and the official provider SDK*. The OpenAI Python SDK [20] handles transport, retries and response types for the 23 OpenAI-compatible providers; a small native adapter handles Anthropic’s Messages API [2], whose OpenAI-compatible endpoint drops prompt caching, thinking output and cached-token usage [3].

Contributions.

- A gateway design (section 3) whose hot path is a table lookup, a cached-client fetch and a call into the provider SDK, and whose responses are the SDK’s own typed objects rather than re-wrapped copies.

- Six techniques (section 4) that remove import, per-call, connection and memory overhead, each tied to the code that implements it.
- An open benchmark suite (section 5) comparing RouteHub, LiteLLM, any-llm and aisuite against the bare provider SDKs. Every library runs in its own environment against an instant mock server, so the numbers isolate client-side cost, plus one real-network experiment on connection reuse.

2 Background and Related Work

Unified LLM clients. LiteLLM [4] is the most widely used gateway. It ships a Python SDK and a proxy server, covers more than 100 providers, and adds cost tracking, callbacks, caching and budget management. Much of that machinery runs inside every `completion()` call. any-llm [16] delegates to each provider’s official SDK and offers both module-level functions and an AnyLLM client class for connection reuse. aisuite [17] is a thin wrapper that loads provider modules by convention. RouteHub sits closest to any-llm in philosophy (lean on official SDKs) and closest to LiteLLM in API surface (drop-in function names and exceptions).

Model routing. “Routing” also names a research problem: choosing *which* model should answer a query to trade cost against quality, as in FrugalGPT’s cascades [5] and RouteLLM’s learned routers [19]. Those systems decide the destination; a gateway like RouteHub is the transport layer they dispatch through. Cheap transport makes fine-grained routing policies cheaper to run.

Startup and tail latency. Python import cost is a known start-up tax: SOCK found that importing popular libraries adds on the order of 100 ms to start-up and built Zygote-based provisioning with package caching to hide it in serverless platforms [18]. At scale, per-request overheads that look negligible at the median surface in the tail [6]. Serving-side work such as vLLM [11] attacks model latency itself and is orthogonal to the client-side overheads studied here.

Software supply chain. Large transitive dependency trees widen the attack surface of every application that installs them [12, 25]. Install-time and start-up-time execution paths such as `.pth` files are especially dangerous because no import is needed to trigger them [15].

3 Design

3.1 Goals

- G1 Near-zero hot path.** Gateway work per call should be small next to the SDK’s own serialisation and HTTP cost.
- G2 Pay for what you use.** Importing the package must not load SDKs, read data files or touch the network.
- G3 SDK-native types.** Return `openai.types.chat` objects so callers and type checkers see the SDK’s real schema.
- G4 No global state.** Retries, TLS, timeouts and parameter dropping are per-call arguments, so tenants sharing a process cannot affect each other.
- G5 Small trusted base.** Three direct dependencies: `openai`, `pydantic` and `tiktoken`.

3.2 Architecture

Figure 1 shows the request path. A call is split once into OpenAI request parameters, gateway options and provider extras (`_split_args`). The model string is resolved against a static table of 24 provider entries by prefix (`groq/llama-3.3-70b`) or by bare-name rules (`claude-*` → Anthropic), and the key and base URL are read from the call or from the provider’s environment variables. The call then takes one of two paths:

- **OpenAI-compatible** (23 entries, including Azure, Gemini, Groq, OpenRouter, Ollama and any `api_base`): a cached `openai.OpenAI` client sends the request, and the SDK’s `ChatCompletion` or stream is returned unchanged.
- **Native Anthropic:** the request is translated to the Messages API, sent through a cached HTTP client, and the response or event stream is translated back into `ChatCompletion` and `ChatCompletionChunk`.

Errors from either path go through one mapping function that raises typed exceptions (`RateLimitError`, `ContextWindowExceededError`, ...) which subclass the matching OpenAI SDK exceptions, so existing handlers keep working.

Listing 1 shows the interface. Every setting is an argument; there is no module-level configuration.

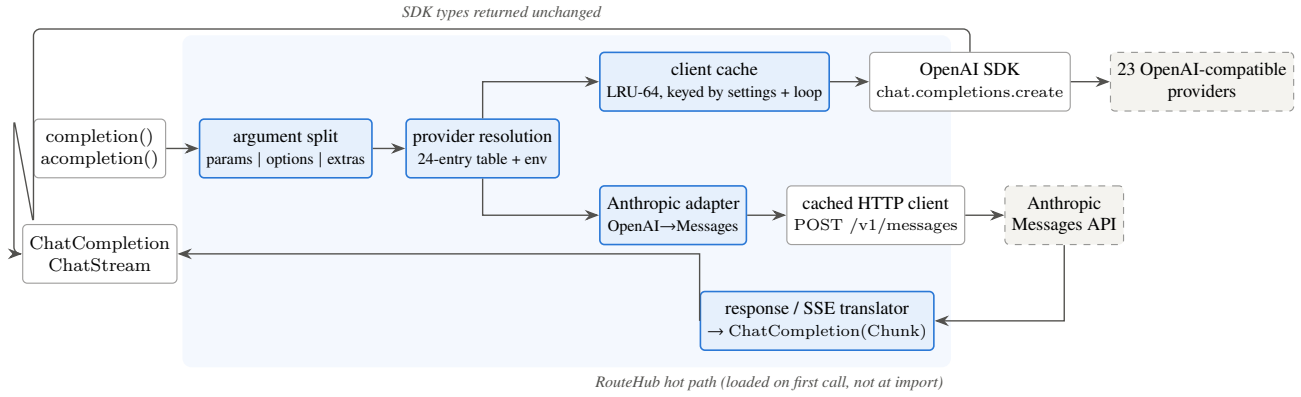


Figure 1: RouteHub request path. Blue boxes are RouteHub code; white boxes are the official OpenAI SDK and HTTP client it delegates to. The OpenAI-compatible path returns the SDK’s own objects; only the Anthropic path translates responses.

```
import routehub

msgs = [{"role": "user", "content": "Summarize the Q3 risks."}]
r = routehub.completion(model="gpt-5.4-mini", messages=msgs)
print(r.choices[0].message.content) # openai ChatCompletion

for chunk in routehub.completion( # native Messages API
    model="claude-sonnet-4-6", messages=msgs,
    tools=[weather], stream=True):
    print(chunk.choices[0].delta.content or "", end="")

r = await routehub.acompletion(
    model="groq/llama-3.3-70b-versatile", messages=msgs,
    num_retries=3, ssl_verify="/etc/ssl/corp-ca.pem")
```

Listing 1: One signature for every provider. Settings are per call.

4 Performance Techniques

4.1 Lazy public API

RouteHub’s package root defines no functions. It maps 47 public names to the submodules that define them and resolves each on first access through a module-level `__getattr__` (PEP 562 [13]), caching the result in the module namespace so later lookups are ordinary attribute reads (listing 2). `import routehub` therefore loads no third-party code. The OpenAI SDK, pydantic and the HTTP library are imported when the first request needs them, and the tokenizer and model catalog only when a caller counts tokens or asks for model metadata.

```
_LAZY = {"completion": "routehub.main",
        "get_model_info": "routehub.registry",
        "RateLimitError": "routehub.exceptions", ...}

def __getattr__(name):
    module = _LAZY.get(name)
    if module is None:
        raise AttributeError(name)
    value = getattr(importlib.import_module(module), name)
    if name not in _UNCACHED: # model_list refreshes
        globals()[name] = value # later reads skip this hook
    return value
```

Listing 2: Lazy attribute resolution in `routehub/__init__.py` (abridged).

The same rule applies at runtime: nothing touches the network until a request does. Model metadata (context windows, prices, capability flags) comes from OpenRouter’s live model list [21], fetched on the first metadata lookup and

cached for five minutes. LiteLLM, by contrast, downloads its model price map from GitHub during import unless `LITELLM_LOCAL_MODEL_COST_MAP` is set [14].

4.2 Thin, allocation-light hot path

The per-call work in front of the SDK is a handful of dictionary operations. `completion()` captures its arguments with `locals()`, splits them once, and resolves the provider with a dictionary lookup and at most two environment reads. Message lists are scanned for response-only fields (`reasoning_content`, `thinking_blocks`) and Anthropic `cache_control` markers, and the original list is passed through *unchanged* when nothing needs removing, so the common case allocates no copies. Responses are not re-validated or re-wrapped: the SDK already parsed them into pydantic models, and RouteHub returns those objects.

To stay forward-compatible without per-call introspection, RouteHub reads the installed SDK’s `create()` signature once per client class and routes parameters the SDK does not yet know into `extra_body` (listing 3). New provider parameters therefore work on older SDKs, and the check costs one set lookup per parameter.

```
_SDK_PARAMS = {} # create() parameter names, per class

def _create_kwargs(completions, request, extra_body, call):
    kind = type(completions)
    if kind not in _SDK_PARAMS:
        _SDK_PARAMS[kind] = frozenset(
            inspect.signature(completions.create).parameters)
    accepted, kwargs = _SDK_PARAMS[kind], {}
    for key, value in request.items():
        if key in accepted: kwargs[key] = value
        else: extra_body.setdefault(key, value)
    kwargs.update(extra_headers=call.extra_headers,
                  extra_body=extra_body or None,
                  timeout=call.timeout)
    return kwargs
```

Listing 3: Fitting a request to the installed SDK (`main.py`, abridged).

4.3 Client pooling keyed by settings and event loop

Building an SDK client constructs an HTTP client, a connection pool and an SSL context, which can take milliseconds. RouteHub keeps up to 64 clients in an LRU cache keyed by everything that changes a client’s behaviour: provider kind, sync or async, API key, base URL, API version, organisation, retry count, TLS setting and keep-alive expiry (listing 4). Async clients are also keyed by the running event-loop *object*, not its `id()`, because async connection pools are bound to one loop and a recycled id could otherwise hand a dead loop’s client to a new one. The lock covers only the dictionary operations, so slow client construction never serialises other threads. Because settings are part of the key, per-call configuration (G4) costs one tuple hash rather than a new client.

```
key = ("openai", provider == "azure", is_async,
      _loop_id(is_async), # loop object, not id()
      api_key, api_base, api_version, organization,
      max_retries, ssl_verify, keepalive_expiry)
return _cached(key, build) # LRU, 64 entries
```

Listing 4: Cache key for SDK clients (`clients.py`).

4.4 Connection reuse across agent steps

HTTP/1.1 persistent connections [8] let a client skip DNS, TCP and TLS set-up [22] on later requests. Both `httpx` and the OpenAI SDK default to closing a pooled connection after 5 s idle [7, 20]. That suits browsers, but an agent that runs a tool for more than five seconds between model calls reconnects at every step. RouteHub builds its pools with a 60-second keep-alive expiry, configurable per call, which covers typical tool latencies. Common load-balancer idle timeouts are 60 s or longer [1], and a connection the server closed first is either detected by the pool or retried by the existing connection-error retry.

4.5 A native, streaming Anthropic adapter

Anthropic’s OpenAI-compatible endpoint is documented as a testing aid: it ignores prompt caching, never returns thinking content, and leaves `prompt_tokens_details` empty [3]. RouteHub calls the Messages API directly. Translation is a single pass in each direction: system messages are hoisted, content parts become typed blocks, tools and `tool_choice` are rewritten, and `reasoning_effort` becomes adaptive thinking with an effort level on models that require it. Streaming uses an incremental server-sent-events parser [23] feeding a small state machine (`StreamTranslator`) that emits one `ChatCompletionChunk` per text, thinking or tool-argument delta, plus a trailing usage chunk on request. The adapter keeps everything the compatibility layer drops: `cache_control` markers, signed thinking blocks, and cache read and write token counts. JSON goes through `orjson` [10] when the optional fast extra is installed.

4.6 Per-call configuration without global state

LiteLLM exposes many settings as module globals: parameter dropping (`litellm.drop_params`), retries (`litellm.num_retries`) and TLS verification (`litellm.ssl_verify`) among them. In a process shared by many agents or tenants, one component’s setting silently changes another’s behaviour. RouteHub accepts every setting as an argument; through the cache key above, two callers with different TLS or retry policies get different pooled clients at no per-call cost.

5 Evaluation

We ask four questions. What does each gateway cost a fresh process (section 5.2)? How much latency does it add to each call (sections 5.3 and 5.5)? How many requests per second can one process push through it (section 5.6)? And does connection reuse matter over a real network (section 5.7)?

5.1 Methodology

Platform. All measurements ran on one Apple M3 Pro laptop (12 cores, 18 GB RAM, macOS 15.7) with CPython 3.12.3 from `python.org` in every environment.

Libraries. RouteHub at commit 55284e1 (openai 3.26.0, httpx 2.2.1, pydantic 2.13.5), with and without `orjson` 3.12.0; LiteLLM 1.104.0 (openai 2.54.0, httpx 0.28.1, aiohttp 3.14.4); `any-llm-sdk` 1.30.0 with its OpenAI and Anthropic extras (openai 3.26.0, anthropic 0.125.0); and `aisuite` 0.2.0 with the same extras (openai 1.109.1). Their pins conflict (`aisuite` requires `openai<2`, `any-llm openai>=2.18`), so each library has its own `uv` virtual environment. Baselines are the bare OpenAI SDK (3.26.0) and Anthropic SDK (1.11.0) using one reused client each; the Anthropic baseline sends a hand-written native request, so it does no translation at all.

Isolation. Every measurement runs in a fresh subprocess whose environment holds only `HOME`, `PATH`, `LANG` and `NO_PROXY`: no API keys and no proxies. LiteLLM runs with `LITELLM_LOCAL_MODEL_COST_MAP=True`, which disables its GitHub download at import and so favours it; one cold-start row shows the default.

Mock server. A single-process `uvloop` server on loopback speaks HTTP/1.1 with keep-alive and returns pre-rendered responses: an OpenAI `ChatCompletion` or Anthropic `Message` with a 60-word reply and usage, or an SSE stream of 200 content deltas sent with chunked transfer encoding. Server time is a few microseconds and identical for every client, so differences between libraries are client-side work.

Workloads. A *1-message* request (one user message, `max_tokens=256`, 154-byte body) and an *agent* request (system prompt, 20 prior turns and a new user message for 22 messages, four function tools with JSON schemas, `temperature=0.2`; 6.7 KB body). Each library is called through its documented API: `completion()` with a provider-prefixed model for RouteHub and LiteLLM; `any_llm.completion()` and an `AnyLLM.create()` client for `any-llm`; `Client().chat.completions` for `aisuite`. `aisuite` 0.2.0 raises `NameError` for any request with tools unless its optional MCP extra is installed, so the harness supplies the missing `is_mcp_config` predicate.

Protocol. Warm runs make 100 untimed calls, then 400 timed calls, three times per configuration with the configuration order rotated between rounds, giving 1,200 samples each. A sample is the wall time of the call plus reading the reply text. Cold runs discard two starts per library (to compile bytecode and warm the file cache), then measure 20 fresh processes per library, interleaved. Streaming runs time 200 streams of 200 chunks per configuration (two rounds of 100); async runs push 1,000 requests through an asyncio semaphore at concurrency 1, 8, 32 and 128. any-llm’s module-level functions appear in the cold and warm results; for streaming and async we use its AnyLLM client (section 5.3).

5.2 Cold start and memory

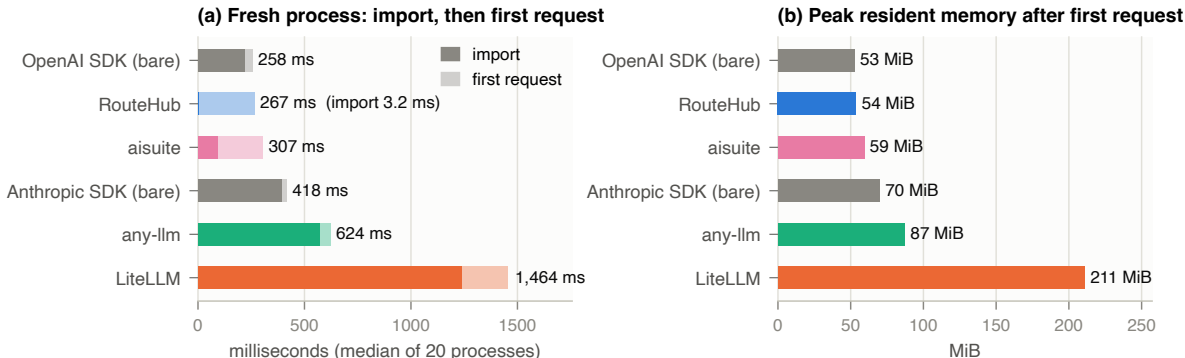


Figure 2: Cost of a fresh process. (a) Median time to import the library (solid) and complete the first request (light), over 20 processes. (b) Peak resident set size after the first request. Gray bars are the bare provider SDKs.

Importing RouteHub takes 3.2 ms and adds 18 entries to `sys.modules`: the package itself and 17 standard-library modules (`re`, `typing`, `enum`, ...), with no third-party code. LiteLLM’s import takes 1,235 ms, 383× longer, and loads 2,502 modules, including 1,040 of its own, the OpenAI SDK, `requests`, `aiohttp`, `jinja2`, `yaml` and `tokenizers`; its process holds 194 MiB before sending anything. any-llm imports both the OpenAI and the Anthropic SDK eagerly (2,163 modules, 569 ms). aisuite loads provider modules on demand and imports in 93 ms.

Because RouteHub defers the SDK to the first request, the fair comparison is import plus first request (fig. 2a): 267 ms for RouteHub against 258 ms for the bare OpenAI SDK, a difference of 9 ms. aisuite needs 307 ms, any-llm 624 ms and LiteLLM 1,464 ms, 5.5× RouteHub. Peak memory after the first request (fig. 2b) follows the same order: 54 MiB for RouteHub, within about a megabyte of the bare SDK (53 MiB), and 211 MiB for LiteLLM, 3.9× as much. With LiteLLM’s default configuration the price-map download adds a further 145 ms to the median import (1,381 ms on our connection), and the import then depends on GitHub being reachable.

Table 1: Cold-start detail (medians of 20 fresh processes). Modules are entries added to `sys.modules`; RSS is peak resident memory.

Library	time (ms)		modules		RSS (MiB)	
	import	1st call	import	+call	import	+call
RouteHub	3.2	264	18	879	13	54
RouteHub + orjson	3.2	269	18	881	13	54
OpenAI SDK (bare)	217.2	40	782	870	48	53
Anthropic SDK (bare)	392.2	25	2,116	2,182	66	70
aisuite	93.3	211	278	1,064	37	59
any-llm (functions)	569.0	55	2,163	2,262	82	87
LiteLLM	1,235.3	218	2,502	3,054	194	211
LiteLLM (default) [†]	1,380.7	195	2,564	3,054	208	222

[†]Without `LITELLM_LOCAL_MODEL_COST_MAP`: the price map is downloaded from GitHub during import.

5.3 Per-call overhead

Figure 3 and table 2 show steady-state latency once clients are warm.

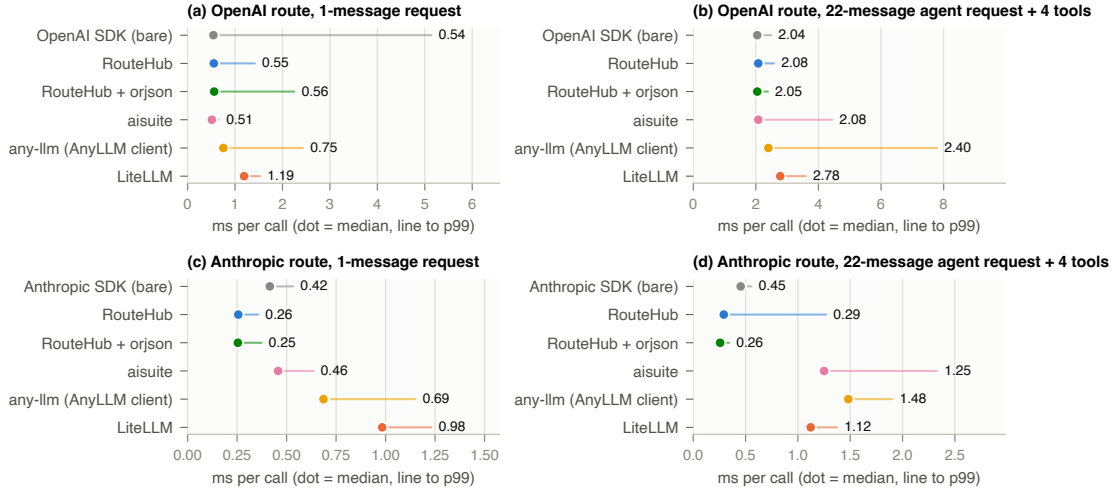


Figure 3: Latency of sequential calls against the instant mock server (1,200 calls per configuration). Dots are medians, lines extend to the 99th percentile. The bare SDK (gray) is the floor: it does the same HTTP and parsing work with no gateway in front.

OpenAI route. The bare SDK needs 0.54 ms for a 1-message call. RouteHub adds +9 μ s, inside run-to-run noise, and +32 μ s for the 22-message agent request. LiteLLM adds +647 μ s and +736 μ s, and any-llm’s AnyLLM client +210 μ s and +354 μ s. aisuite, which also hands requests straight to the SDK, is as cheap as RouteHub on this route. (The bare SDK in the LiteLLM, any-llm and aisuite environments measured 0.52, 0.52 and 0.50 ms, so SDK version differences are small next to the gateway differences.) LiteLLM’s extra time is work it does on every call: it creates a logging object, runs cache and response-metadata hooks, converts the reply into its own response type, and submits a success handler to a background thread pool (section 5.4).

Anthropic route. Here RouteHub is *faster than the official SDK*: 0.26 ms against 0.42 ms for one message, and 0.29 ms against 0.45 ms for the agent request. That is despite RouteHub also translating the OpenAI-format request into Messages format and the response back. The adapter encodes the body once, sends it on a pooled HTTP client and validates the reply straight into a ChatCompletion; the SDK builds typed request and response models of its own. aisuite and any-llm call the Anthropic SDK and then convert its result, paying for both. For the agent request they add +797 μ s and +1,029 μ s over the SDK, and LiteLLM adds +669 μ s.

Per-call clients. any-llm’s module-level completion() builds a provider client for every call. That costs +1,031 μ s on the OpenAI route and 13.16 ms per call on the Anthropic route, where constructing an Anthropic SDK client dominates. Each discarded client also holds its connection until it is garbage-collected. In our runs this produced occasional 5.5 s stalls (the SDK’s 5 s connect timeout plus a retry) that put the 99th percentile at 5.5 s. The AnyLLM client, which any-llm recommends for production, avoids both problems, so it is the any-llm configuration plotted.

Table 2: Median sequential call latency in ms (1,200 calls each); the 99th percentiles are drawn in fig. 3. The OpenAI baseline is openai 3.26.0.

Library	OpenAI route		Anthropic route	
	1 message	agent	1 message	agent
Provider SDK (bare)	0.54	2.04	0.42	0.45
RouteHub	0.55	2.08	0.26	0.29
RouteHub + orjson	0.56	2.05	0.25	0.26
aisuite	0.51	2.08	0.46	1.25
any-llm (AnyLLM client)	0.75	2.40	0.69	1.48
any-llm (functions)	1.58	3.20	13.16	11.99
LiteLLM	1.19	2.78	0.98	1.12

5.4 Where the time goes

Wall-clock overheads of tens of microseconds are close to the noise floor, so we also count work deterministically. Table 3 profiles 200 warm calls per library with cProfile and reports the Python function calls each request executes and the share of profiled self-time spent in the gateway’s own package. RouteHub adds +70 calls to the bare SDK’s 5,961 for a 1-message request, and its own code accounts for 1.7% of the time. LiteLLM adds +4,615 calls, and its own package is 26.7% of the time.

The agent request also exposes a cost inside the SDK itself. The bare OpenAI SDK executes 35,644 function calls to send the 22-message request, against 5,961 for one message. 38% of the time goes to the SDK’s own code, which walks and transforms every message and tool definition before serialising. RouteHub inherits that cost on the OpenAI route (+264 calls of its own), which is why its agent-request latency there is 2.08 ms while its native Anthropic path, which serialises the body itself, sends the same 22 messages in 0.29 ms. Bypassing the SDK’s request transform is the largest remaining optimisation on RouteHub’s OpenAI path.

Table 3: Work per request under cProfile (OpenAI route, 200 warm calls). Δ is relative to the bare OpenAI SDK 3.26.0; aisuite and LiteLLM run on the SDK versions they pin (1.109.1 and 2.54.0).

Library	request	fn calls	Δ	own %
aisuite	1 msg	5,588	-373	0.6
OpenAI SDK (bare)	1 msg	5,961	—	—
RouteHub	1 msg	6,031	+70	1.7
any-llm (AnyLLM client)	1 msg	6,815	+854	1.9
LiteLLM	1 msg	10,576	+4,615	26.7
OpenAI SDK (bare)	agent	35,644	—	—
RouteHub	agent	35,908	+264	0.8
LiteLLM	agent	41,726	+6,082	13.2

5.5 Streaming

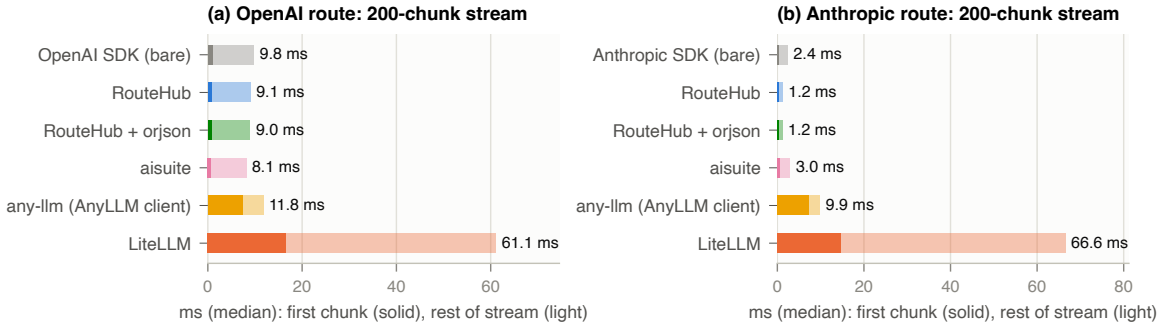


Figure 4: Time to consume a 200-chunk stream from the instant mock server: first content chunk (solid) and the rest of the stream (light), medians of 200 streams. On the Anthropic route every library except the bare SDK also translates each event into an OpenAI chunk.

Streaming multiplies per-event cost by the number of chunks (fig. 4). On the OpenAI route RouteHub wraps the SDK’s iterator in a thin error-mapping shim and drains a 200-chunk stream in 9.1 ms, against 9.8 ms for the bare SDK. On the Anthropic route RouteHub parses the event stream and builds every OpenAI chunk itself, yet drains the stream in 1.2 ms, about half the 2.4 ms the Anthropic SDK needs for its own native events (5 against 10 μ s per chunk). LiteLLM delivers its first chunk after 16.61 ms and drains the stream in 61.1 ms on the OpenAI route (14.69 and 66.6 ms on the Anthropic route), about 224 μ s per chunk. any-llm’s client takes 7.42 ms to its first chunk on both routes. At 100 chunks per second, a 0.2 ms per-chunk cost is 2% of a core for every open stream, so a process fanning out to 50 concurrent streams spends a full core on chunk handling alone.

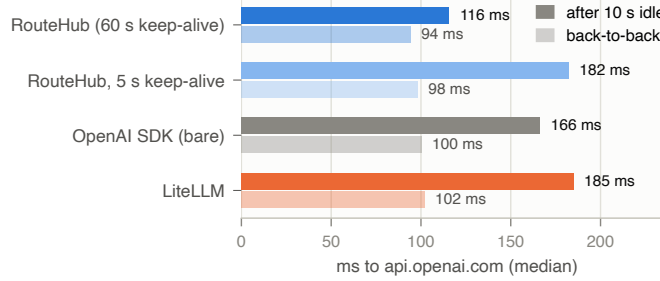


Figure 5: Real network: requests to api.openai.com after a 10 s idle gap (solid) and immediately after (light). Medians of 6 repetitions. Only RouteHub’s 60 s keep-alive reuses the connection across the gap.

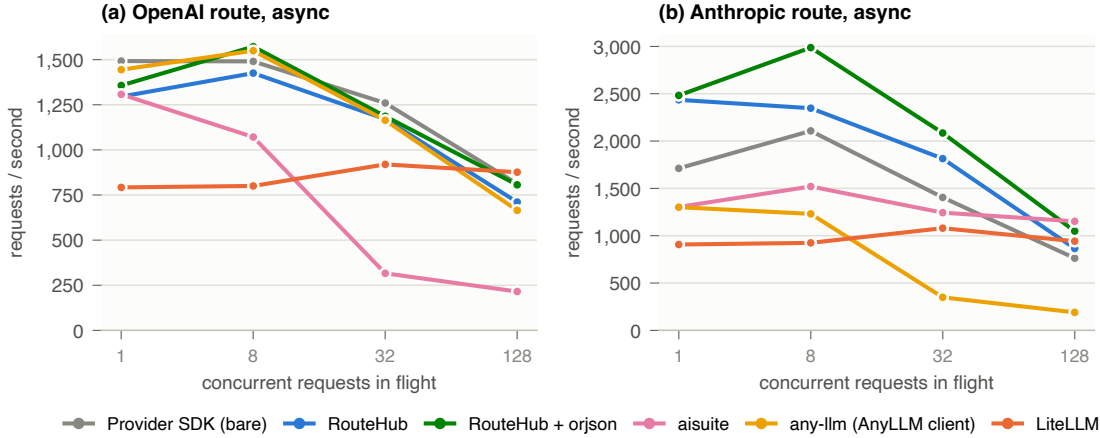


Figure 6: Requests per second from one Python process using each library’s async API (1,000 requests per point, 1-message workload).

5.6 Concurrent throughput

Figure 6 reports throughput from one process. At low concurrency it mirrors per-call cost. On the OpenAI route RouteHub serves 1,295 requests per second against the bare SDK’s 1,492 and LiteLLM’s 792. On the Anthropic route RouteHub reaches 2,435 requests per second against 1,711 for the Anthropic SDK and 907 for LiteLLM.

As concurrency grows, every httpx-based client slows down, the bare SDKs included, so the limit is the shared HTTP stack rather than any gateway’s own code. LiteLLM sends async requests through an aiohttp transport by default, and its throughput stays flat. At 128 requests in flight it overtakes the httpx-based clients: 876 requests per second on the OpenAI route against RouteHub’s 711 and the bare SDK’s 812, and 942 on the Anthropic route against RouteHub’s 865 (1,048 with orjson). aisuite and any-llm’s client fall as low as 215 and 190 requests per second on some routes. Each point is a single run of 1,000 requests, and neighbouring lines within about 10% should not be ranked. For RouteHub, the result shows that its transport, not its gateway code, bounds throughput at high fan-out. Both provider SDKs ship an aiohttp backend (DefaultAioHttpClient) that RouteHub could expose as a per-call option.

5.7 Connection reuse over a real network

Loopback hides connection set-up: it has no round trips and no TLS. To measure reuse we sent requests to api.openai.com with a deliberately invalid key. The provider answers 401 at once, so the timing is connection work plus one round trip, with no model time and nothing billed. After one request opened a connection, each client slept 10 s, sent a timed request, then sent another immediately, 6 times.

Back-to-back requests take 94–102 ms for every client (fig. 5). After the pause, RouteHub with its default 60 s keep-alive reuses its connection and answers in 116 ms. The same code with a 5 s expiry, the bare OpenAI SDK and LiteLLM all reconnect and take 182, 166 and 185 ms. The 67 ms difference is a TCP and TLS handshake to the provider, paid at every agent step whose tool call outlasts five seconds; over a 50-step agent run it adds about 3.3 s of pure connection set-up. These figures come from one client location, and the absolute savings scale with the round-trip time to the provider.

5.8 Dependency footprint

Table 4: Installed footprint of a clean environment containing only the library (with the extras needed for OpenAI and Anthropic).

Library	packages	MiB on disk	.py files
RouteHub	21	22.9	2,271
aisuite	19	16.7	2,539
any-llm	26	28.0	4,198
LiteLLM	58	168.5	5,140

Table 4 counts what each library installs. RouteHub’s three direct dependencies resolve to 21 packages (23 MiB); LiteLLM installs 58 packages and 168 MiB, including boto3, huggingface_hub, tokenizers, aiohttp and jinja2. aisuite is smaller still (19 packages), though it holds applications to the older openai 1.x line. Each installed package is code that runs with the application’s credentials and a release channel that can be compromised; LiteLLM’s March 2026 incident [15] showed that a .pth file can execute on every interpreter start without the package ever being imported.

6 Discussion and Limitations

What the mock server does and does not show. The loopback server removes network and model time so that client cost is visible. In production a request also spends tens of milliseconds on the network and hundreds to thousands generating tokens, so for a single interactive call the gateway’s share is small for every library tested. The differences matter when they are multiplied: by the number of steps in an agent run, the number of processes started, or the number of requests one process must sustain. Throughput (section 5.6) is where client CPU becomes the bottleneck, because a Python process can only spend each core-second once.

Feature parity. LiteLLM does far more than RouteHub: a proxy server, budgets, spend logging, caching, callbacks, routers and fallbacks across more than 100 providers. Part of its per-call cost pays for that machinery, and users who need it inside the client may accept the trade. RouteHub’s position is that those features belong outside the per-request hot path, in the application or a separate service, and that the default client should cost no more than the provider SDK it wraps.

Threats to validity.

- *One machine and OS.* Results come from one Apple-silicon laptop on macOS. Absolute numbers will differ on Linux servers; we expect the ordering to hold because the dominant costs (modules imported, objects built per call) are platform-independent.
- *Different SDK versions.* Pins force different OpenAI SDK versions per environment, so we report each environment’s bare-SDK baseline alongside the gateway (section 5.3).
- *API choice.* any-llm’s module-level functions build a provider client per call, which its documentation positions for simple scripts; its AnyLLM client is the production path. We report both. aisuite needed a one-line shim to accept tools at all.
- *Author bias.* The benchmark was written by RouteHub’s developers. To limit this, every library uses its documented API, baselines bound what is achievable, configuration order is rotated, and the harness and raw samples are published with the paper.

Lazy loading is not free. Deferring imports moves cost from import to the first request (fig. 2). The total is what matters, and RouteHub’s total is close to the bare SDK’s. A process that imports the gateway but never sends a request (a CLI printing --help, a test that mocks the model) avoids the SDK import entirely.

7 Conclusion

RouteHub shows that a multi-provider gateway can cost little more than the provider SDK it wraps. Three choices do most of the work: load nothing until a request needs it, hand back the SDK’s own response objects instead of re-wrapping them, and keep clients and their connections alive across calls and across the pauses of an agent loop. None of these is exotic.

Each removes work from the request path rather than making that work faster, and together they leave the gateway with little to do beyond a table lookup, a cache hit and a call into code the provider already maintains.

The measurements bear this out at every stage of a request's life. A process that imports RouteHub loads 18 modules instead of LiteLLM's 2,502, reaches its first response $5.5\times$ sooner and needs $3.9\times$ less peak memory, finishing within 9 ms of the bare OpenAI SDK. Once warm, RouteHub adds +9 μ s to an OpenAI-route call where LiteLLM adds +647 μ s, and it executes +70 Python function calls beyond the SDK's own against LiteLLM's +4,615. On Anthropic's API RouteHub's native adapter is faster than the official SDK: 0.26 ms against 0.42 ms per call and 5 against 10 μ s per streamed chunk, while preserving the prompt caching, thinking output and cache accounting that the OpenAI-compatible endpoint discards. Over a real network its 60 s keep-alive saves 67 ms on every call that follows a pause of more than five seconds, and the whole library installs 21 packages where LiteLLM installs 58.

For agent workloads these savings compound. An agent that takes 50 steps, each separated by a tool call of ten seconds or more, avoids roughly 3.3 s of connection set-up and a few tens of milliseconds of gateway CPU. It also starts more than a second sooner when it runs in a fresh worker. A system that fans out to many concurrent streams keeps its cores for application work rather than per-chunk bookkeeping. None of this changes model latency, which still dominates any single request. It changes what the gateway contributes to every request, and that contribution is paid again on every step, process start and stream.

We draw five lessons for gateway design. First, *delegate to the provider SDKs*: they already implement retries, timeouts, pooling and typed responses, and every layer re-implemented on top of them is a layer that must be paid for and kept in sync. Second, *make import free*: lazy attribute resolution costs a dictionary lookup and turns a fixed tax on every process into a cost paid only by processes that send requests. Third, *put configuration in the cache key, not in globals*: per-call settings then cost one tuple hash, and tenants sharing a process cannot affect each other. Fourth, *tune the transport for the workload*: HTTP defaults assume browsers, while agents pause for seconds between calls, and a single keep-alive setting recovers a handshake per step. Fifth, *measure work, not only time*: function-call counts per request are deterministic and machine-independent, and they expose overheads that are lost in the noise of wall-clock timing.

The benchmarks also point to RouteHub's next steps. The largest remaining cost on the OpenAI route is the SDK's own recursive request transform, which RouteHub could bypass by serialising the body itself, as its Anthropic path already does. At high fan-out the shared httpx transport, rather than gateway code, bounds throughput, so exposing the SDKs' aiohttp backend as a per-call option is a natural extension. The project's open issues add HTTP/2 as an optional transport, the OpenAI Responses API, provider batch APIs, and native Bedrock and Vertex AI adapters. Each is to be built the same way, outside the per-request hot path, so that the property measured here, a gateway that costs about as much as the SDK beneath it, holds as RouteHub's surface grows. It already keeps LiteLLM's call signature, so for most applications adopting it is a change of import.

Reproducibility

The benchmark suite, raw results and figure scripts are in `paper/benchmarks/` next to this paper. `run_all.py` setup builds the per-library environments with `uv`; `run_all.py all` and `run_all.py idle` regenerate every result; `make_figures.py` redraws every figure and table.

References

- [1] Amazon Web Services. Edit attributes for your Application Load Balancer: Connection idle timeout. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/edit-load-balancer-attributes.html>, 2026.
- [2] Anthropic. Messages API reference. <https://platform.claude.com/docs/en/api/messages>, 2026.
- [3] Anthropic. OpenAI SDK compatibility. <https://platform.claude.com/docs/en/cli-sdks-libraries/libraries/openai-sdk>, 2026.
- [4] BerriAI. LiteLLM: Python SDK and proxy server (LLM gateway) to call 100+ LLM APIs in OpenAI format. <https://github.com/BerriAI/litellm>, 2026. Version 1.104.0.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [6] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.
- [7] Encode. HTTPX: Resource limits. <https://www.python-httpx.org/advanced/resource-limits/>, 2026.
- [8] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP/1.1. <https://www.rfc-editor.org/rfc/rfc9112>, 2022.
- [9] Kye Gomez and The Swarm Corporation. RouteHub: A fast, lightweight LLM gateway with a litellm-compatible API. <https://github.com/The-Swarm-Corporation/RouteHub>, 2026. Commit 55284e1, accessed 2026-10-07.
- [10] ijl. orjson: Fast, correct Python JSON library. <https://github.com/ijl/orjson>, 2026.

- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [12] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. SoK: Taxonomy of attacks on open-source software supply chains. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [13] Ivan Levkivskiy. PEP 562 – module `__getattr__` and `__dir__`. <https://peps.python.org/pep-0562/>, 2017.
- [14] LiteLLM. Custom model cost map. https://docs.litellm.ai/docs/proxy/custom_model_cost_map, 2026.
- [15] LiteLLM. Security update: Suspected supply chain incident. <https://docs.litellm.ai/blog/security-update-march-2026>, March 2026. Published 2026-03-24.
- [16] Mozilla.ai. any-llm: Communicate with an LLM provider using a single interface. <https://github.com/mozilla-ai/any-llm>, 2026. Package any-llm-sdk, version 1.30.0.
- [17] Andrew Ng et al. aisuite: Simple, unified interface to multiple generative AI providers. <https://github.com/andrewyng/aisuite>, 2026. Version 0.2.0.
- [18] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. SOCK: Rapid task provisioning with serverless-optimized containers. In *USENIX Annual Technical Conference (ATC)*, 2018.
- [19] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations (ICLR)*, 2025.
- [20] OpenAI. The official Python library for the OpenAI API. <https://github.com/openai/openai-python>, 2026. Versions 1.109.1, 2.54.0 and 3.26.0.
- [21] OpenRouter. List available models. <https://openrouter.ai/api/v1/models>, 2026.
- [22] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. <https://www.rfc-editor.org/rfc/rfc8446>, 2018.
- [23] WHATWG. HTML living standard: Server-sent events. <https://html.spec.whatwg.org/multipage/server-sent-events.html>, 2026.
- [24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [25] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small world with high risks: A study of security threats in the npm ecosystem. In *USENIX Security Symposium*, 2019.